



Comprehensions & Looping Idioms

Python Mastery — Part 1: Foundations · Week 8

Rathinam Trainers & Consultants

Live on Microsoft Teams · recorded

Recap & today

Where we are in the journey

Last week — Sets & Dictionaries

- ▶ **Sets** for uniqueness: `|` `&` `-` `^`, and the `set(...)` dedupe
- ▶ **Dicts** keyed by id: `get`, `.items()`, add & remove keys
- ▶ Choosing the container: **list** vs **set** vs **dict**
- ▶ `collections`: `Counter`, `defaultdict`, `namedtuple`

You can now model tasks properly. Today we learn to **say it in one line**.

Today's goal

By the end of this session you'll understand how to:

- ▶ Write **list**, **set**, and **dict** comprehensions
- ▶ Filter with `if` and transform with `a if cond else b`
- ▶ Use `enumerate`, `zip`, `reversed`, `sorted(key=...)`
- ▶ Judge when a comprehension **helps** — and when it hurts

Comprehensions

Saying "build me a list" in one line

The loop you keep writing

You have written this shape a dozen times already:

```
squares = []  
for n in range(1, 6):  
    squares.append(n * n)
```

Three lines of **bookkeeping** for one idea:

"the square of every n ".

Python has a way to say just the idea.

The list comprehension

```
squares = [n * n for n in range(1, 6)]  
#          ^^^^^ ^      ^^^^^^^  
#          what  each   from where
```

- ▶ Read it **left to right**: *what* you want, *for each* item, *from* where
- ▶ The brackets `[ ]` say: the result is a **list**
- ▶ Same result as the loop — `[1, 4, 9, 16, 25]`

Adding a filter

Put an `if` at the **end** to keep only some items:

```
evens = [n for n in range(10) if n % 2 == 0]  
# [0, 2, 4, 6, 8]
```

The long way, for comparison:

```
evens = []  
for n in range(10):  
    if n % 2 == 0:  
        evens.append(n)
```

A trailing `if` **filters**: items that fail are dropped.

Transforming with a conditional expression

`a if cond else b` is an **expression** — it produces a value:

```
labels = ["even" if n % 2 == 0 else "odd" for n in range(5)]  
# ['even', 'odd', 'even', 'odd', 'even']
```

Nothing is dropped — every item is **transformed**.

The position rule

Same word, two jobs — **position** decides which:

```
[n for n in xs if n > 0]          # filter  -> at the END  
[n if n > 0 else 0 for n in xs]  # choose  -> at the FRONT
```

- ▶ **if** **after** the **for** → **filter**: keep or drop
- ▶ **if ... else** **before** the **for** → **transform**: always produce a value
- ▶ A filter has **no** **else**; a transform **must** have one

Set comprehensions

Swap the brackets for braces — you get a **set** (unique, unordered):

```
tags = {tag for t in tasks.values() for tag in t["tags"]}
```

- ▶ Duplicates collapse automatically
- ▶ Print order is **not** guaranteed — wrap in `sorted(...)` to display

Dict comprehensions

Braces **plus a colon** — you get a **dict**:

```
titles = {tid: t["title"] for tid, t in tasks.items()}  
# {1: 'Buy milk', 2: 'Write report', ...}
```

- ▶ `key: value` before the `for`
- ▶ `.items()` gives you both halves to work with

Nested comprehensions

Two `for` clauses read in the same order as nested loops:

```
flat = [n for row in matrix for n in row]
# outer first ^^^^^^^^^^^^^ ^^^^^^^^^^^^^ then inner
```

```
for row in matrix:           # the same thing, unrolled
    for n in row:
        flat.append(n)
```

Left to right, outer to inner. Get this backwards → `NameError`.

Let's shrink some real loops.

Watch this — Demo 15

Rewrite loops as comprehensions

- ▶ A `for` loop that filters → a comprehension with `if`
- ▶ A **conditional expression** to label each task
- ▶ A **dict** comprehension `{id: title}`
- ▶ A **set** comprehension of every tag
- ▶ Which loop variables **leak** — and which don't
- ▶ A **nested** one, and the order that breaks it

Loop → comprehension

```
>>> open_titles = []
>>> for t in tasks.values():
...     if not t["done"]:
...         open_titles.append(t["title"])
...
>>> open_titles
['Buy milk', 'Call plumber']

>>> [t["title"] for t in tasks.values() if not t["done"]]
['Buy milk', 'Call plumber']
```

Four lines become one. Same answer.

Dict & set comprehensions

```
>>> {tid: t["title"] for tid, t in tasks.items()}  
{1: 'Buy milk', 2: 'Write report', 3: 'Call plumber', 4: 'Archive photos'}
```

```
>>> sorted({tag for t in tasks.values() for tag in t["tags"]})  
['home', 'shopping', 'urgent', 'work']
```

The set collapsed every duplicate tag on its own.

What you just saw

- ▶ A filtering loop collapsed to **one readable line**
- ▶ A trailing `if` **drops** items; `if/else` up front **transforms** them
- ▶ `{k: v for ...}` builds a **dict**; `{x for ...}` builds a **set**
- ▶ The set **deduplicated the tags** on its own
- ▶ A comprehension's variable **never leaks** — a plain `for`'s **does**
- ▶ Nested `for`s read **left to right, outer to inner** — or `NameError`

Looping idioms

The helpers that replace bookkeeping

enumerate — index and value

Don't count by hand:

```
n = 1                                # DON'T
for t in tasks.values():
    print(n, t["title"])
    n += 1
```

```
for n, t in enumerate(tasks.values(), start=1): # DO
    print(n, t["title"])
```

- ▶ `start=1` because humans count from 1. Default is `0`
- ▶ Note `.values()` — iterating a **dict** gives you its **keys**

`zip` — walk two lists together

```
for title, day in zip(titles, due_dates):  
    print(f"{title:<14} due {day}")
```

- ▶ Pairs them up item by item
- ▶ Stops at the **shortest** — extras are silently ignored
- ▶ `zip(a, b, strict=True)` raises `ValueError` if lengths differ

reversed — back to front

```
for name in reversed(names):  
    print(name)
```

- ▶ Gives you a **view**, not a new list — wrap in `list(...)` to see it
- ▶ The original list is **untouched**
- ▶ `list(reversed([1, 2, 3]))` → `[3, 2, 1]`
- ▶ `.reverse()` (the method) **mutates** instead — same trap as `.sort()`

`sorted(key=...)` — sort by anything

`key` takes a **function** — it says *what to sort on*:

```
sorted(tasks.values(), key=lambda t: t["title"])      # by title
sorted(words, key=str.lower)                          # case-insensitively
sorted(tasks.values(), key=lambda t: (t["done"], t["title"])) # two keys
```

- ▶ A **tuple** key sorts by the first, then breaks ties with the second
- ▶ `reverse=True` flips it
- ▶ Python's sort is **stable** — equal keys keep their original order

`sorted()` vs `.sort()`

```
>>> nums = [3, 1, 2]
>>> sorted(nums)
[1, 2, 3]
>>> nums
[3, 1, 2]          # untouched – sorted() RETURNS a new list

>>> nums.sort()    # returns None!
>>> nums
[1, 2, 3]          # mutated in place
```

`sorted()` copies · `.sort()` mutates and returns `None`

Generator expressions — a peek

Round brackets: same shape, but **lazy**:

```
sum(n * n for n in range(1, 6))    # 55 – no list ever built
```

- ▶ A list comprehension builds **everything**, now
- ▶ A generator produces items **one at a time, on demand**
- ▶ Perfect for feeding `sum`, `max`, `any` — full story in **Week 15**

Let's present the task list properly.

Watch this — Demo 16

`enumerate` / `zip` / `sorted(key=...)`

- ▶ Number the list with `enumerate(tasks, start=1)`
- ▶ `zip` titles against due dates — and `strict=True`
- ▶ `sorted(key=lambda t: t["title"])`, then a **two-key** sort
- ▶ `reversed`, and `sorted()` vs `.sort()`

enumerate & zip

```
>>> for n, t in enumerate(tasks.values(), start=1):  
...     print(f"{n}. {t['title']}")  
...  
1. Buy milk  
2. Write report  
3. Call plumber  
4. Archive photos  
  
>>> list(zip(titles, due, strict=True))  
ValueError: zip() argument 2 is shorter than argument 1
```

No counter variable. And `strict=True` catches the mismatch.

Sorting by a key

```
>>> sorted(["banana", "apple", "Cherry"])  
['Cherry', 'apple', 'banana']
```

```
>>> sorted(["banana", "apple", "Cherry"], key=str.lower)  
['apple', 'banana', 'Cherry']
```

Capital **C** sorts before lowercase **a** — until **key** says
"compare them lowercased".

What you just saw

- ▶ `enumerate` numbered the list — **no counter variable**
- ▶ `zip` walked two lists in step; `strict=True` catches mismatches
- ▶ `key=` changed the **sort order without touching the data**
- ▶ A **tuple key** sorted by done-ness, then alphabetically
- ▶ `sorted()` handed back a new list; `.sort()` mutated and returned `None`

When *not* to

Readability beats cleverness

The readability rule

```
# Fine – one idea, reads like a sentence  
[t["title"] for t in tasks if not t["done"]]
```

```
# Stop. Write the loop.  
[f(x, y) for x in xs if p(x) for y in g(x) if q(x, y)]
```

- ▶ If you can't read it **out loud in one breath**, use a loop
- ▶ A comprehension is for **building a collection** — nothing else
- ▶ Side effects (printing, saving) belong in a **real loop**

Lab 8 — make the task app idiomatic

1. **Refactor** your Lab 6/7 loops into comprehensions where they genuinely read better
2. **Number** the displayed list with `enumerate(..., start=1)`
3. **Sort** by title *and* by done-ness with `sorted(key=...)`
4. Behaviour unchanged · `uvx ruff check` clean
5. **Submit:** `tasks.py` + sample output to the Teams channel

Mirror **Demo 15** for the comprehensions, **Demo 16** for the sorts.

Next week

- ▶ **Week 9 — Modules & Packages**
- ▶ Our task app stops being one file and becomes a **package**
- ▶ `import`, the `__name__ == "__main__"` guard, and `python -m taskapp`

See you in the lab.

Questions?